



UNIVERSIDADE DA CORUÑA

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN COMPUTACIÓN



Diseño e implementación de un sistema de navegación volumétrica para agentes voladores en videojuegos

Estudiante: Javier Manotas Ruiz
Dirección: Enrique Fernández Blanco
Alejandro Puente Castro

Introducción

Fundamentos

Metodología y planificación

Desarrollo

Resultados

Conclusiones y trabajos futuros

Introducción

Fundamentos

Metodología y planificación

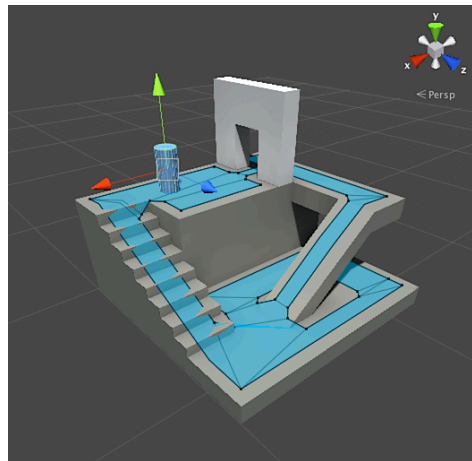
Desarrollo

Resultados

Conclusiones y trabajos futuros

El punto de partida: la navegación autónoma

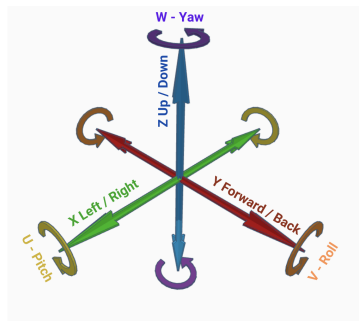
- La navegación autónoma es la base de la inteligencia de los personajes no jugables (NPCs).
- El *pathfinding* está prácticamente resuelto para agentes **terrestres**.
- Los motores integran soluciones nativas basadas sobre la biblioteca Recast.
- Se apoyan en una **mall de navegación 2.5D**: el mundo es 3D, pero las rutas se calculan sobre superficies donde el agente se apoya.



Malla 2.5D sobre un entorno 3D.

Motivación: los agentes voladores

- Drones, naves, criaturas o peces no están anclados al suelo: operan con **seis grados de libertad**.
- Un plano transitable es insuficiente. Se necesita un **volumen de navegación** que represente **todo** el espacio libre.
- Añadir la altura como dimensión libre dispara el coste en tiempo y memoria.
- Sin soporte oficial, los estudios recurren a soluciones *ad-hoc*.



Los seis grados de libertad.

Necesidad

Un sistema **genérico, optimizado y reutilizable** que resuelva la navegación volumétrica.

Objetivo general: diseño, implementación y validación en Unity de un sistema de navegación volumétrica eficiente y optimizado, con **enfoque de producto**.

- Arquitectura modular y escalable.
- Representación compacta del espacio (estructura de datos espacial).
- Serialización en disco del volumen.
- Búsqueda de rutas heurística en 3D.
- Postprocesado y suavizado de trayectorias.
- Integración nativa en el editor de Unity.
- Accesibilidad para perfiles técnicos y artistas.
- Análisis cuantitativo del rendimiento.
- Distribución como paquete estándar de Unity.

Introducción

Fundamentos

Metodología y planificación

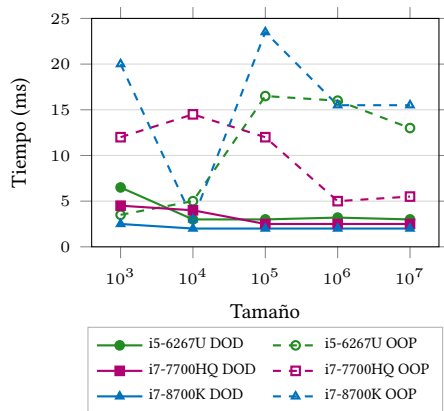
Desarrollo

Resultados

Conclusiones y trabajos futuros

Fundamentos tecnológicos: Unity y alto rendimiento

- **Unity**: arquitectura de componentes (`GameObject` + `MonoBehaviour`) y *game loop*.
- **Editor**: atributos y ventanas personalizables.
- **Físicas (PhysX)**: la geometría se consulta con `OverlapBox`. Se puede paralelizar.
- **DOTS / diseño orientado a datos**: `Job System`, **Burst** (LLVM + SIMD), `Unity.Mathematics` y colecciones nativas.
- **Bajo nivel en C#**: `ref readonly`, `Span<T>`, `stackalloc` y manipulación de bits.

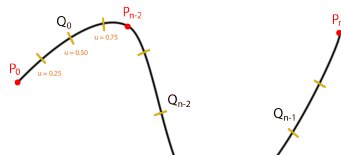


El diseño orientado a datos aprovecha mejor la caché que la POO.

- **Representación del espacio:** frente a *waypoints* y *flow fields*, se elige el **Sparse Voxel Octree (SVO)**, que solo subdivide donde hay geometría.
- **Códigos de Morton** (curvas de orden Z): mapean coordenadas 3D a un índice lineal preservando la localidad espacial.
- **Búsqueda:** de Dijkstra a A^* , con $f(n) = g(n) + h(n)$.
- **Postprocesado:** atajo por línea de visión (DDA) y suavizado con *splines* de Catmull-Rom.
- **Evación:** ORCA, sobre el espacio de velocidades.
- **Integridad:** *hash* determinista FNV-1a.

	00	01	10	11
00	0000	0001	0100	0101
01	0010	0011	0110	0111
10	1000	1001	1100	1101
11	1010	1011	1110	1111

Curva de Morton (versión 2D).



Introducción

Fundamentos

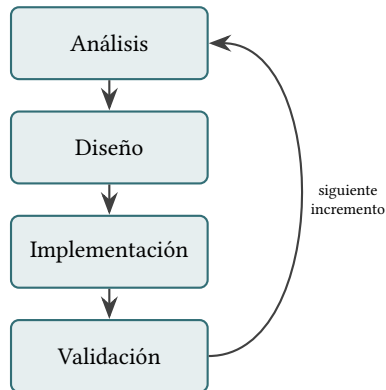
Metodología y planificación

Desarrollo

Resultados

Conclusiones y trabajos futuros

- Metodología **iterativa e incremental**: el sistema avanza por incrementos funcionales en lugar de un bloque monolítico.
- Cada iteración recorre cuatro fases: **análisis, diseño, implementación y validación**.
- Punto de partida: un **producto mínimo viable** centrado en la viabilidad técnica.
- Código separado en módulos mediante *assemblies* y validación automática con integración continua desde el primer día.



Planificación inicial

Tarea / Fase	Marzo		Abril				Mayo				Junio			
Semana	1	2	3	4	5	6	7	8	9	10	11	12	13	14
T1. Estado del arte y estudio previo														
T2. Diseño de la arquitectura														
T3. Estructura de datos volumétrica														
T4. Búsqueda y postprocesado de rutas														
T5. Integración en Unity														
T6. Evaluación y optimización														
T7. Validación y pruebas (CI)														
Redacción de la memoria														
Hitos			H1				H2			H3			H4	H5

- 360 h (créditos ECTS), 14 semanas.
- Fases T1–T7 con cinco hitos (H1–H5).

Coste estimado del proyecto

Concepto	Coste
Desarrollo (360 h)	4.114,80 €
Tutorías (30 h)	572,10 €
Amortización equipo	87,50 €
Licencias	0,00 €
Indirectos (5 %)	210,12 €
Total	4.984,52 €

Tarea / Fase	Marzo		Abril				Mayo				Junio				Jul.
Semana	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
T1. Estado del arte y estudio previo															
T2. Diseño de la arquitectura															
T3. Estructura de datos volumétrica															
T4. Búsqueda y postprocesado de rutas															
Extensión T4. Robustez geométrica (<i>clipping</i>)															
T5. Integración en Unity															
T6. Evaluación y optimización															
T7. Validación y pruebas (CI)															
T8 (extra). Evasión local															
Redacción de la memoria															
Hitos			H1				H2				H3		H4	H5	

Coste final del proyecto

Concepto	Coste
Desarrollo (400 h)	4.572,00 €
Tutorías (30 h)	572,10 €
Amortización equipo	87,50 €
Licencias	0,00 €
Indirectos (5 %)	232,98 €
Total	5.464,58 €

- La robustez geométrica (*clipping*) alargó la fase T4.
- La holgura permitió añadir evasión local (T8).
- Proyecto final: 15 semanas y **400 h** reales.

Introducción

Fundamentos

Metodología y planificación

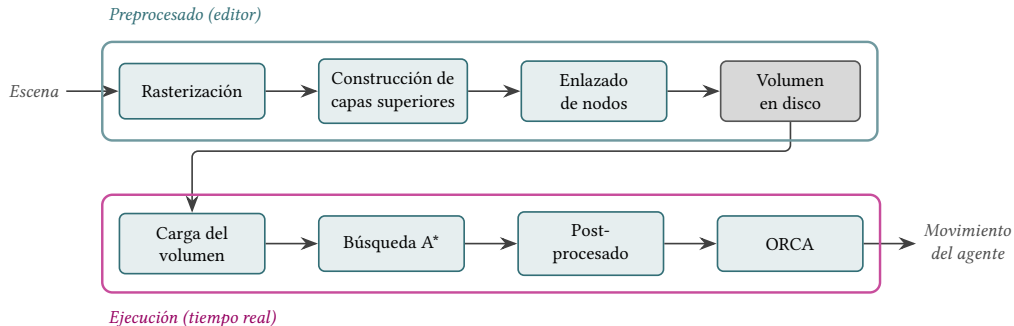
Desarrollo

Resultados

Conclusiones y trabajos futuros

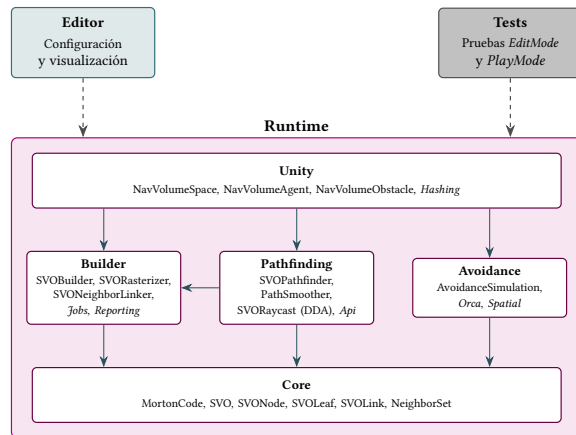
Visión global del sistema

El sistema final es un *pipeline* con dos fases bien diferenciadas:



El **preprocesado** asume a propósito el mayor coste (se ejecuta una vez en el editor). La **ejecución** prioriza el rendimiento con estructuras compactas y paralelización.

- Tres bloques separados físicamente como *assemblies*:
 - **Runtime**: lógica de navegación (único en la versión final).
 - **Editor**: herramientas de configuración y visualización.
 - **Tests**: pruebas *EditMode* y *PlayMode*.
- *Editor* y *Tests* dependen de *Runtime*, nunca a la inversa.
- *Runtime* crece de forma controlada sobre una base común, *Core*.



Iteración 1: estructura de datos volumétrica (SVO)

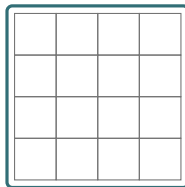
Representación compacta

- SVO en capas planas, ordenadas por código de Morton (búsqueda binaria).
- Cada hoja es una rejilla $4 \times 4 \times 4$ codificada en una **máscara de 64 bits** (8 bytes).
- **Punteros compactos** de 32 bits para referenciar nodos y vóxeles.

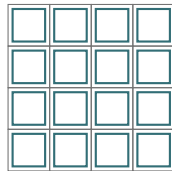
Construcción (*bake*)

- Muestreo con `OverlapBox` paralelizado (Job System + Burst).
- Estrategia **de lo grueso a lo fino** y descarte temprano de hojas vacías.
- Serialización en un único *blob* binario + *hash* FNV-1a de integridad.

Descarte de hojas vacías:



Pasada gruesa:
1 consulta



Pasada fina:
64 consultas

Reduce las consultas físicas en un factor cercano a 64
en la mayor parte del volumen.

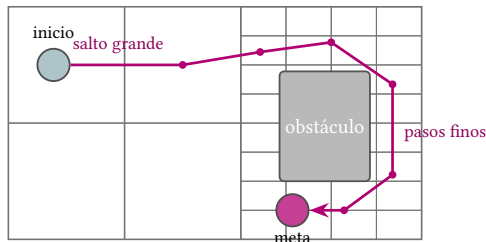
Iteración 2: búsqueda A* y postprocesado

A* sobre un grafo de resolución heterogénea

- Un solo salto cruza grandes nodos de espacio abierto.
- Cerca de los obstáculos, la búsqueda se ramifica en nodos y vóxeles cada vez más pequeños.
- Dos modos de coste: distancia euclídea o conteo de nodos, con heurística ponderada
 $f(n) = g(n) + W h(n)$.

Postprocesado (seguro por construcción)

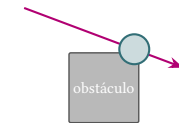
- Atajo voraz por **línea de visión** (algoritmo DDA).
- Suavizado con *spline* de Catmull-Rom, que pasa exactamente por los puntos ya verificados como libres.



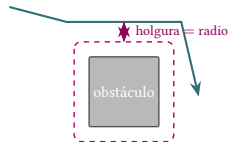
A* avanza a saltos grandes en el espacio abierto y se afina junto al obstáculo.

Iteración 3: robustez geométrica y radio del agente

- **Problema (*clipping*):** las rutas rozaban la geometría porque el agente se trataba como un punto sin volumen.
- **Solución:** **inflar** los obstáculos por el radio del agente durante la rasterización. Cualquier ruta libre conserva, por construcción, la holgura necesaria.
- Sin fase adicional: basta con expandir las cajas de consulta `OverlapBox`.
- Esta desviación obligó a una batería de pruebas mayor (*PlayMode* + CI), que destapó defectos sutiles (p. ej. un desbordamiento $1 << 40$ corregido con $1UL << 40$).



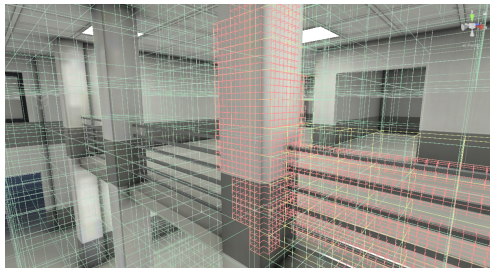
Sin radio: la trayectoria recorta la esquina



Obstáculo inflado: trayectoria con holgura

Iteración 4: integración en Unity

- Componente central NavVolumeSpace con tres modos de construcción: *Baked*, *BuildOnAwake* y *Manual*.
- **API** de navegación y consultas espaciales: comprobar si un punto es navegable, ajustarlo al vóxel libre más cercano o generar posiciones aleatorias válidas.
- Rutas **síncronas** y **asíncronas** (reserva de buscadores y cancelación por *token*), sin bloquear el hilo principal.
- **Gizmos** de depuración visual y panel de estadísticas (memoria estimada y % de ahorro).



Visualización del SVO con Gizmos: vóxeles, hojas y nodos.

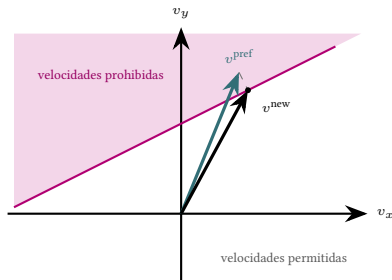
Iteraciones 5 y 6: optimización y evasión local

Iteración 5: optimización

- Reutilización de *buffers* entre peticiones.
- Comprobación de cancelación amortizada.
- Acceso por referencia (`ref readonly`) y comparaciones sin *boxing*.
- *Bake* más rápido al evitar reordenar capas innecesariamente.

Iteración 6: evasión local (ORCA)

- Evita colisiones entre agentes mediante reciprocidad sobre el espacio de velocidades.
- `stackalloc + Span<T>`: cero presión sobre el recolector de basura.
- *Hash* espacial y *jobs* asíncronos (Burst).
- Respeta también la geometría estática del entorno.



Introducción

Fundamentos

Metodología y planificación

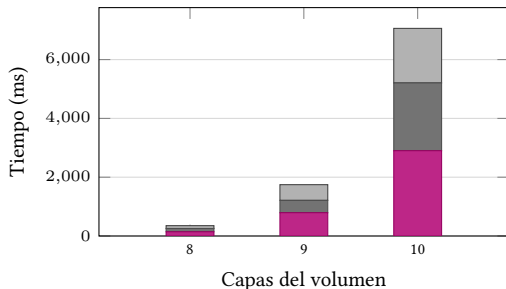
Desarrollo

Resultados

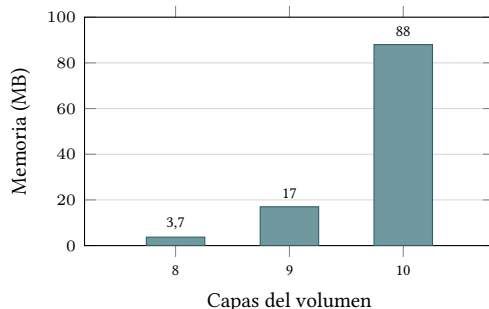
Conclusiones y trabajos futuros

Rendimiento: construcción y memoria del volumen

Escenario más desfavorable (interior de un edificio de varias plantas); media de 30 ejecuciones en un Intel Core i7-12700H.



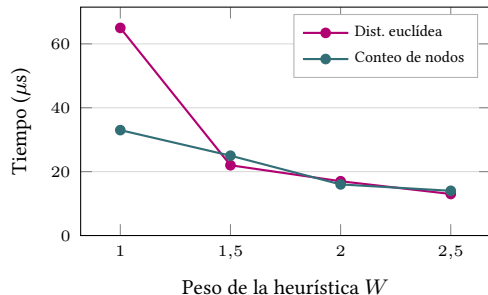
Tiempo de *bake*: 0,35 s (8) a 7,1 s (10 capas).



Ahorro frente a una rejilla densa: >96 % (8) y >98 % (9-10 capas).

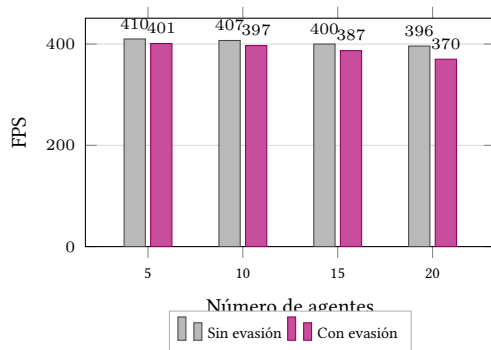
La rasterización de hojas y el guardado en disco concentran ~70 % del coste de construcción.

Rendimiento: búsqueda de rutas y evasión



Una ruta larga: 65 μs a 13 μs , muy por debajo de los 16,6 ms de un fotograma a 60 FPS.

El sistema cumple los requisitos de eficiencia sin comprometer la tasa de fotogramas.



Con 20 agentes, la evasión añade $\sim 0,18$ ms (menos de 10 μs por agente).

Introducción

Fundamentos

Metodología y planificación

Desarrollo

Resultados

Conclusiones y trabajos futuros

Objetivo general alcanzado

No es un mero algoritmo de búsqueda, sino un **sistema completo**: de la geometría de la escena al movimiento final de un agente que esquiva obstáculos estáticos y a otros agentes.

Logros

- Todos los objetivos específicos cumplidos.
- Funcionalidad extra no planificada: la evasión local entre agentes.
- Paquete modular listo para producción.

Limitaciones

- En 3D no se garantiza el camino óptimo (se acota con heurística y límite de nodos).
- El volumen se precalcula: no apto para geometría dinámica en tiempo de ejecución.
- Máximo de 10 capas (Morton de 32 bits, 1024 celdas por eje).

- **Construcción del volumen en la GPU** mediante *compute shaders*: muy paralelizable, aunque de prioridad baja porque el *bake* no es el cuello de botella.
- **Búsqueda y evasión con aprendizaje por refuerzo**: sustituir los enfoques reactivos y deterministas (A^* + ORCA) por comportamientos más naturales y eficientes.
- **Otras líneas de extensión**:
 - Geometría dinámica y reconstrucción incremental del SVO.
 - Búsqueda jerárquica o de cualquier ángulo (*any-angle*).
 - Enlaces entre volúmenes independientes para escenarios de gran tamaño.



UNIVERSIDADE DA CORUÑA

Gracias por su atención

¿Preguntas?

Javier Manotas Ruiz • Navegación volumétrica para agentes voladores en videojuegos